Online accompaniement for "Collaborative and parametric insurance on the Ethereum blockchain"

Pierre-Olivier Goffard [*1] and Stéphane Loisel[†2]

[1] Université de Strasbourg, Institut de Recherche Mathématique Avancée, Strasbourg, France

[2] CNAM, Paris, France

June 30, 2025

We will review the Solidity code of the smart contract discussed in the paper titled *Collaborative and Parametric Insurance on the Ethereum Blockchain*. The core mechanism of our parametric insurance is implemented in the contract `InsuranceLogic.sol`. It relies on two helper contracts: `PricingLogic.sol`, which encodes the pricing formula, and `ModelPointsLogic.sol`, which performs the model point aggregation for the portfolio. We will first examine the helper contracts, `PricingLogic.sol` in Section 1 and `ModelPointsLogic.sol` in Section 2. We will then explore the methods of the main smart contract in Section 3.

1 `PricingLogic.sol`

We start by indicating the version of the compiler and the name of the smart contract in code fragment 1.

Solidity is a high-level programming language designed for smart contract development, where instructions are terminated with semicolons. The language is continuously evolving, with each stable release tied to a specific compiler version. This versioning ensures backward compatibility, allowing developers to compile code written in older Solidity versions and maintain stability for existing contracts. The following code fragments are intended to populate the space between the brackets following the contract declaration. In Code Fragment 2, we declare several variables.

[*] Email: goffard@unistra.fr.

[†] Email: stephane.loisel@lecnam.net.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.19;

contract PricingLogic {
    ---
}
```

Code Fragment 1: Declaring a compiler and define `PricingLogic` contract.

```
    uint256 public constant factor = 10 ** 4; // Numerical precision
    address public owner; // Ethereum address of the owner of the contract


    // Define a struct to hold polynomial coefficients
    struct PolyCoeffs {
        int256 c0;
        int256 c1;
        int256 c2;
        int256 c3;
        int256 c4;
    }

    // Mapping from station name to polynomial coefficients
    mapping(string => PolyCoeffs) public stationCoeffs;
```

Code Fragment 2: Variable declaration of the `PricingLogic.sol` contract.

When declaring a variable in Solidity, one must specify its type, visibility, and name. A variable of type `address` stores a 20-byte value that represents the location of accounts (both externally owned accounts and smart contracts) on the Ethereum network. In Solidity, numbers can only be stored as integer values in variables of type `uint`x, where x specifies the bit length of the number. Other basic types include strings, booleans, and fixed-size byte arrays[1]. A `struct` is a custom data type that can store various data fields; in this context, it is used to store the coefficients of the polynomial used to smooth the probability of the triggering event as a

[1] See https://docs.soliditylang.org/en/latest/types.html

function of the day of the year. Mappings in Solidity function as data dictionaries. Here, we create a mapping to retrieve the coefficients of the polynomial associated with the probability of triggering events in locations such as Marseille or Strasbourg. The constructor method in 3 initializes the contract's data.

```
constructor() {
    owner = msg.sender;

    // Initialize coefficients for "MARSEILLE-MARIGNANE"
    stationCoeffs["MARSEILLE-MARIGNANE"] = PolyCoeffs(
       61866466488584768,    // 6.08088853e-02 * 1e11
        1279067435587755,     // 1.30072825e-03 * 1e11
        -22081841812020,      // -2.22982823e-05 * 1e11
        109525424150,         // 1.10635770e-07 * 1e11
        -160548258           // -1.62373694e-10 * 1e11
    );

    // Initialize coefficients for "STRASBOURG-ENTZHEIM"
    stationCoeffs["STRASBOURG-ENTZHEIM"] = PolyCoeffs(
        76579341005774688,     // 7.53896117e-02 * 1e11
        -322886972669433,       // -2.16506423e-04 * 1e11
        13084236350914,        // 1.17177460e-05 * 1e11
        -66687593698,         // -6.10554025e-08 * 1e11
        91781331              // 8.44071936e-11 * 1e11
    );
}
```

Code Fragment 3: Constructor method of `PricingLogic`.

We set the value of the coefficients associated to the polynomial approximation of the triggering event probability in Marseille and Strasbourg. The conract only have one mehod called `getProbability` taht takes as argument `T` the day of the year and `station` which indicates the location. The code is provided in code fragment 4.

This method returns the probability that the level of precipitation exceeds 5 mm on a particular day and at a particular location. This method will be called within the `InsuranceLogic` contract to provide a quote to a customer.

```
    function getProbability(uint256 T, string memory station) public view returns (uint)
         {
        // Retrieve the coefficients for the given station
        PolyCoeffs memory coeffs = stationCoeffs[station];

        // // Scale T by 1e10 for fixed-point arithmetic
        int256 scaledT = int256(T);
        int result = coeffs.c4; // Start with the highest degree coefficient

        // // Evaluate the polynomial using Horner's method
        result = result * scaledT + coeffs.c3;
        result = result * scaledT + coeffs.c2;
        result = result * scaledT + coeffs.c1;
        result = result * scaledT + coeffs.c0;

        // Return the result scaled down by 1e18
        return uint256(result * int256(factor) / 1e18);
    }
```

Code Fragment 4: `getProbability` method of the `PricingLogic` contract.

2 ModelPointsLogic.sol

The second auxiliary contract is called `ModelPointsLogic`; see code fragment 5.

```
pragma solidity 0.8.19;

contract ModelPointsLogic {
    ---
}
```

Code Fragment 5: Declaring a compiler and define `ModelPointsLogic` contract.

The goal of this smart contract is to monitor our parametric insurance portfolio by keeping track of the mean, variance, and skewness of the liability distribution. The liability of the parametric insurance portfolio corresponds to the payout associated with model points that represent a group of contracts. The variables of

ModelPointsLogic are given in code fragment 6.

```
address public owner; // Ethereum address of the owner of the contract
uint256 public constant factor = 10 ** 4; // Numerical precision
uint256 public mu = 0;
uint256 public sn2 = 0;
uint256 public gamma1_unormalized = 0; // Skewness times std to the power 3 of the
    liability
uint256 public Nt = 0 ; // Counter for the number of insurance contracts
uint256 public Nt_MP = 0; // Counter for the number of model points ever created
uint256 public Nt_MP_active = 0; // Counter for the number of active model points
```

Code Fragment 6: Variables of the ModelPointsLogic contract.

The variable mu is the mean of the liability, the variable sn2 is the variance of the liability, and gamma1_unormalized is the skewness of the liability times the standard deviation to the power of 3/2. This allows us to simply sum the unnormalized skewness coefficients associated with each model point. We keep track of the number of parametric insurance contracts, the number of model points ever created, and, more importantly, the number of active model points, which are those for which the event date is still ahead of us. Note that the number of active model points must be higher than a threshold (5 in our implementation) in order to use the normal approximation to calculate the solvency capitals. In code fragment 7, we define two data structures to store the information of parametric insurance contracts and model points, respectively.

Sets of parametric insurance contracts and model points are then indexed using mappings. The constructor method is not particularly interesting for the ModelPointsLogic contract, and so we omit it here. Code fragments 8, 9, and 10 show the addInsuranceContract method.

When a new parametric insurance contract is added to the portfolio, the portfolio of model points is updated along with the moments of the liability. This information is then forwarded to the InsuranceLogic contract, which calculates the solvency capital requirements. We also have a method to remove a policy from the portfolio, as it can be closed or canceled. The Solidity code is provided in code fragments 11 and 12.

```
// Define a struct to represent an insurance contract
struct InsuranceContract {
    string station;
    uint256 T; // day in a year
    uint256 p; // probability
    uint256 l; // compensation
    string status; // status of the contract

}

// // Define a struct to represent a model point
struct ModelPoint {
    string station;
    uint256 T; // day in a year
    uint256 p; // probability
    uint256 l_sum; // Sums of the compensations
    uint256 mean; // mean of the Model Point
    uint256 variance; // variance of the Model Point
    uint256 skew_unormalized; // skewness times std to the power 3 of the Model
        Point
}

// Mapping from uint to an array of InsuranceContract
mapping(uint256 => InsuranceContract) public insuranceContracts;

// Mapping from uint to an array of ModelPoint
mapping(uint256 => ModelPoint) public modelPoints;
```

Code Fragment 7: Data structures of the `ModelPointsLogic` contract.

As the smart contract may undergo a reset if the surplus falls below the Minimum Capital Required, all the remaining active policies are canceled. The portfolio then needs to be updated, which is possible with the `cancelAllActiveContracts` method of code fragment 13.

```
function addInsuranceContract(string memory station, uint256 T, uint256 p, uint256 l
    ) public {
    // Calculate mean, variance, and skewness of the payout
    uint256 mean;
    uint256 variance;
    uint256 skew_unormalized;

    // Increment the contract counter
    Nt++;
    // Add the new insurance contract to the mapping
    insuranceContracts[Nt] = InsuranceContract({
        station: station,
        T: T,
        p: p,
        l: l,
        status: "active"
    });
```

Code Fragment 8: Part 1 of `addInsuranceContract` method.

3 InsuranceLogic.sol

The core mechanisms of our parametric insurance solution on the blockchain are implemented in `InsuranceLogic`. We start by declaring the dependencies of our contract with respect to `PricingLogic` and `ModelPointsLogic` in code fragment 14.

This is done using interfaces in which we also indicate the state variables and methods from `PricingLogic` and `ModelPointsLogic` that we will be using. We then declare our smart contract together with its variables in code fragment 15.

We keep track of the total number of protocol tokens, the exchange of tokens against ETH, and the surplus. We have the initial values of the parameters corresponding to the safety loading and the risk level associated with the solvency capitals. We monitor the number of parametric insurance contracts as well as the number of model points. The moments of the liability are derived from the information provided by `ModelPointsLogic`. Code fragment 16 defines a data structure to store the information associated with a parametric insurance

```
// Check if a model point with the same T and station exists
bool modelPointExists = false;
for (uint256 i = 1; i <= Nt_MP; i++) {
    if (keccak256(abi.encodePacked(modelPoints[i].station)) == keccak256(abi.
        encodePacked(station)) && modelPoints[i].T == T) {
        // Update the mean, variance and skewness of the liability
        if (modelPoints[i].l_sum == 0) {
            Nt_MP_active++;
        }
        mu = mu - modelPoints[i].mean;
        sn2 = sn2 - modelPoints[i].variance;
        gamma1_unormalized = gamma1_unormalized - modelPoints[i].
            skew_unormalized;
        // Update the mean, variance, and skew of the existing model point
        uint256 l_sum = modelPoints[i].l_sum + l;
        modelPoints[i].l_sum = l_sum;
        mean = (p * l_sum) / factor;
        variance = (p * (factor - p)* l_sum * l_sum) / (factor * factor);
        skew_unormalized = p * l_sum * l_sum * l_sum / factor - 3 * mean *
            variance - mean * mean * mean;
        modelPoints[i].mean = mean;
        modelPoints[i].variance = variance;
        modelPoints[i].skew_unormalized = skew_unormalized;
        mu = mu + modelPoints[i].mean;
        sn2 = sn2 + modelPoints[i].variance;
        gamma1_unormalized = gamma1_unormalized + modelPoints[i].
            skew_unormalized;
        modelPointExists = true;
        break;
    }
}
```

Code Fragment 9: Part 2 of addInsuranceContract method.

```
        // If no model point exists, create a new one
        if (!modelPointExists) {
            Nt_MP++;
            Nt_MP_active++;
            mean = (p * l) / factor;
            variance = (p * (factor - p) * l * l) / (factor * factor);
            skew_unormalized = p * l * l * l / factor - 3 * mean * variance - mean *
                mean * mean;

            modelPoints[Nt_MP] = ModelPoint({
                station: station,
                T: T,
                p: p,
                l_sum: l,
                mean: mean,
                variance: variance,
                skew_unormalized: skew_unormalized

            });
            mu = mu + mean;
            sn2 = sn2 + variance;
            gamma1_unormalized = gamma1_unormalized + skew_unormalized;
        }
    }
```

Code Fragment 10: Part 3 of `addInsuranceContract` method.

contract.

We further define a mapping to retrieve the policies in our portfolio. Surplus providers lock funds in the smart contract in exchange for protocol tokens, as shown in code fragment 17.

Since `InsuranceLogic` is the smart contract with which users primarily interact, the actions taken by users must be monitored. This is done using events defined in code fragment 18.

```
function removeInsuranceContract(uint256 index, string memory newStatus) public {
    // require(msg.sender == owner, "Only the owner can remove contracts");
    require(index > 0 && index <= Nt, "Invalid contract index");
    require(
        keccak256(abi.encodePacked(insuranceContracts[index].status)) == keccak256(
            abi.encodePacked("active")),
        "Contract is not active"
    );
    require(
        keccak256(abi.encodePacked(newStatus)) == keccak256(abi.encodePacked("
            cancelled")) ||
        keccak256(abi.encodePacked(newStatus)) == keccak256(abi.encodePacked("closed
            ")),
        "Invalid status"
    );

    // Update the status of the contract
    insuranceContracts[index].status = newStatus;

    // Retrieve the contract details
    string memory station = insuranceContracts[index].station;
    uint256 T = insuranceContracts[index].T;
    uint256 l = insuranceContracts[index].l;
```

Code Fragment 11: Part 1 of `removeInsuranceContract` method.

Users interact with the smart contract via methods, and each method is associated with an event. An event is emitted every time a method is called, meaning that the event data is recorded on the blockchain.

In order to use methods from `PricingLogic` and `ModelPointsLogic`, we must instantiate them and include them in the `constructor` of `InsuranceLogic`, as shown in code fragment 19.

A `receive` method, see code fragment 20, must be added to enable the smart contract to collect funds from investors as well as premiums from policyholders.

The contract owner can change the parameters of the parametric insurance scheme. These parameters include

the premium loading and the risk level of the solvency capitals. The Solidity code of the `updateParameters` method is provided in code fragment 21.

A change in the order of the quantile of the normal distribution used to calculate the Value-at-Risk implies an update of the MCR and SCR.

The surplus provider locks funds in the smart contract in exchange for protocol tokens. This mechanism is encoded in the `fundContract` method shown in code fragment 22.

We store the address of the surplus provider in case they are a newcomer, and we further update the token holdings and the total supply of tokens as new units are minted. The converse operation consists of burning protocol tokens and withdrawing ETH. This can be done via the `withdraw` method provided in code fragment **??**.

Before buying coverage from our smart contract, customers can ask for a quote using the `getQuote` method.

The premium calculation relies on the `getProbability` method of `PricingLogic`. The commercial premium is then obtained after applying a loading to the pure premium. The customer, equipped with a quote, can then underwrite a contract using the `underwritePolicy` method. The code is provided in the code fragments 25 and 26.

The portfolio of contracts, model points, and the statistics of the liability are updated accordingly, hence the interaction with `ModelPointsLogic` through the method `addInsuranceContract`. The insurance agreement can only be concluded if the surplus is higher than the SCR; otherwise, the contract gets canceled right away, as shown in code fragment 26.

Once the triggering event date is reached, the contract owner inputs the value of the parameter for that date. We would like to automate this task using a data oracle, but for now we rely on the contract owner to settle the insurance agreement. The code is provided in code fragments 27 and 28.

We again have interactions with `ModelPointsLogic`, except that we use the `removeInsuranceContract` method. A claim is paid if the observable quantity is greater than 5. It is then possible that the surplus falls below the MCR, in which case the smart contract must be reset using the `refundPremiumToPolicyholders` and `redistributeToInvestors` methods described later on.

The `updateReserveAfterSettlement` method updates the surplus depending on whether a claim has been paid or not; see code fragment 29.

When the smart contract must undergo a reset, the first step is to refund the policyholders with active policies. This is done via the `refundPremiumToPolicyholders` method of code fragment 30.

The second step is to redistribute the remaining funds to the surplus providers, which is done using the `redistributeToInvestors` method in code fragment 32.

We have implemented our own square root function using the ancient Babylonian method. The code is provided in code fragment 32.

The helper function `sendPayout` in code fragment 33 ensure that ETH transfers from smart contract to policyholders or investors are possible.

Lastly, the helper function `stringToBytes32` transforms a character string into a sequence of 32 bytes, which is the common representation of a word in Solidity. The code is provided in code fragment 34.

A word can be arbitrarily long, which can be problematic when stored in an event. It is therefore possible to store it under the `bytes32` format so that its length becomes predictable.

```
        // Update the corresponding model point
        for (uint256 i = 1; i <= Nt_MP; i++) {
            if (keccak256(abi.encodePacked(modelPoints[i].station)) == keccak256(abi.
                encodePacked(station)) && modelPoints[i].T == T) {
                mu = mu - modelPoints[i].mean;
                sn2 = sn2 - modelPoints[i].variance;
                gamma1_unormalized = gamma1_unormalized - modelPoints[i].
                    skew_unormalized;

                // Update the model point's l_sum, mean, and variance
                modelPoints[i].l_sum -= l;
                if (modelPoints[i].l_sum == 0) {
                    Nt_MP_active--;
                    modelPoints[i].mean = 0;
                    modelPoints[i].variance = 0;
                    modelPoints[i].skew_unormalized = 0;
                } else {
                    uint256 mean = (modelPoints[i].p * modelPoints[i].l_sum) / factor;
                    uint256 variance = (modelPoints[i].p * (factor - modelPoints[i].p) *
                         modelPoints[i].l_sum * modelPoints[i].l_sum) / (factor * factor)
                        ;
                    uint256 skew_unormalized = modelPoints[i].p * modelPoints[i].l_sum *
                         modelPoints[i].l_sum * modelPoints[i].l_sum / factor - 3 * mean
                        * variance - mean * mean * mean;
                    modelPoints[i].mean = mean;
                    modelPoints[i].variance = variance;
                    modelPoints[i].skew_unormalized = skew_unormalized;
                    mu = mu + mean;
                    sn2 = sn2 + variance;
                    gamma1_unormalized = gamma1_unormalized + skew_unormalized;
                }
                break;
            }
        }
    }
```

Code Fragment 12: Part 2 of removeInsuranceContract method.

```
function cancelAllActiveContracts() public {
        for (uint256 i = 1; i <= Nt; i++) {
            if (keccak256(bytes(insuranceContracts[i].status)) == keccak256(bytes("
                active"))) {
                removeInsuranceContract(i, "cancelled");
            }
        }
    }
```

Code Fragment 13: cancelAllActiveContracts method.

```
pragma solidity 0.8.19;

interface IPricingLogic {
    function getProbability(uint256 T, string memory station) external view returns (
        uint);
}
interface IModelPointsLogic {
    function sn2() external view returns (uint256);
    function gamma1_unormalized() external view returns (uint256);
    function Nt_MP_active() external view returns (uint256);
    function addInsuranceContract(string memory station, uint256 T, uint256 p, uint256 l
        ) external;
    function removeInsuranceContract(uint256 index, string memory newStatus) external;
    function cancelAllActiveContracts() external;
}
```

Code Fragment 14: Dependency declaration of InsuranceLogic to PricingLogic and ModelPointLogic .

```
contract InsuranceLogic{
    address public owner; // Ethereum address of the owner of the contract
    uint256 public constant factor = 10 ** 4; // Numerical precision
    uint256 public totalSupply; // Total supply of tokens
    uint256 public rt = 10000; // Exchange rate of token against ETH
    uint256 public Xt; // Reserve to calculate the exchange rate
    uint16 public eta = 1000; // 10% loading (1000 basis points) to reward the token
        holders
    uint16 public qAlphaSCR = 25758; // standard normal quantile of order alpha = 0.995
    uint16 public qAlphaMCR = 10364; // standard normal quantile of order alpha = 0.995
    uint256 public Nt; // Counter for the number of insurance contracts
    uint256 public Nt_MP_active; // Solvency Capital Requirement
    uint256 public SCR; // Solvency Capital Requirement
    uint256 public MCR; // Minimum Capital Requirement
    uint256 public mu = 0; // mean of the liability distribution
    uint256 public sn = 0; // std of the liability distribution
    uint256 public gamma1 = 0; // Skewness of the liability distribution
    uint256 public l_tot = 0; // Sum of all the potential payouts
    uint256 public Nt_MP_active_min = 5; // Minimum number of model points to trigger
        SCR and MCR
```

Code Fragment 15: Contract and variable declaration of InsuranceLogic.

```
 struct InsuranceContract{
        address customer; // Address of the policyholder
        uint256 T; // Day of the year between 1 and 365
        bytes32 station; // Description of the insured event
        uint256 l; // Payout amount
        uint256 p; // Probability of the event
        uint16 eta; // loading of the premium at the underwritting time
        uint256 refund; // Refund amount in case of bankruptcy
        uint8 status; // Status of the contract 0 = open, 1 = closed and settled, 2 =
            closed without compensation, 3 = refunded
    }
    mapping(uint256 => InsuranceContract) public insuranceContracts; // Mapping of
        insurance contracts
```

Code Fragment 16: InsuranceContract data structure

```
address[] public investorAddresses; // Array to store investor addresses
mapping(address => uint256) public Yt; // Balance of investors in tokens
```

Code Fragment 17: Array of investorAddresses and token holding

```
event Fund(address indexed from, uint256 x, uint256 y);
event Burn(address indexed from, uint256 x, uint256 y);
event ParametersUpdated(uint16 newEta, uint16 newQAlphaSCR, uint16 newQAlphaMCR, uint256
     newSCR, uint256 newMCR);
event InsuranceUnderwritten(uint256 indexed contractId, address indexed customer,
    uint256 T, bytes32 station, uint256 l, uint256 cp, uint8 status, uint256 newSCR,
    uint256 newMCR);
event ClaimSettled(uint256 indexed contractId, address indexed customer, bool
    payoutTransferred, uint256 newSCR, uint256 newMCR);
```

Code Fragment 18: Event definition of InsuranceLogic

```
IPricingLogic public pricingLogic;
    IModelPointsLogic public modelPointsLogic;

    constructor(address _pricingLogicAddress, address _modelPointsLogicAddress) {
        owner = msg.sender;
        pricingLogic = IPricingLogic(_pricingLogicAddress);
        modelPointsLogic = IModelPointsLogic(_modelPointsLogicAddress);
    }
```

Code Fragment 19: Constructor of InsuranceLogic

```
receive() external payable {}
```

Code Fragment 20: Receive method of InsuranceLogic

```
function updateParameters(uint16 newEta, uint16 newQAlphaSCR, uint16 newQAlphaMCR)
    public {
    require(msg.sender == owner, "Only the owner can update parameters");
    eta = newEta;
    qAlphaSCR = newQAlphaSCR;
    qAlphaMCR = newQAlphaMCR;
    if (Nt_MP_active >= Nt_MP_active_min) {

        SCR = sn * qAlphaSCR / factor + sn * gamma1 * qAlphaSCR * qAlphaSCR / factor /
            factor / 6 - sn * gamma1 / 6 - mu;
        MCR = sn * qAlphaMCR / factor + sn * gamma1 * qAlphaMCR * qAlphaMCR / factor /
            factor / 6 - sn * gamma1 / 6 - mu;
    }
    if (Xt < MCR) {
            refundPremiumToPolicyholders();
            if (Xt > 0){
                redistributeToInvestors();
            }
            SCR=0;
            MCR=0;
            sn=0;
            mu=0;
            gamma1=0;
            rt=10000;
            Xt=0;
            Nt_MP_active=0;
            modelPointsLogic.cancelAllActiveContracts();
        }

    emit ParametersUpdated(newEta, newQAlphaSCR, newQAlphaMCR, SCR, MCR);
    }
```

Code Fragment 21: updateParameters method of InsuranceLogic

```
function fundContract() public payable {
        require(msg.value > 0, "Funding amount must be greater than 0");
        uint256 tokensToMint = (msg.value * (factor)) / rt;
        totalSupply += tokensToMint;
        Xt += msg.value;
        if (Yt[msg.sender] == 0) {
            investorAddresses.push(msg.sender);
            }
        Yt[msg.sender] += tokensToMint;
        emit Fund(msg.sender, msg.value, tokensToMint);
    }
```

Code Fragment 22: fundContract method of InsuranceLogic

```
function withdraw(uint256 _value) public returns (bool success) {
        require(Yt[msg.sender] >= _value, "Insufficient balance");
        uint256 etherAmount = _value * rt / (factor);
        require(Xt - SCR >= etherAmount, "Insufficient contract balance");

        Yt[msg.sender] -= _value;
        totalSupply -= _value;
        Xt -= etherAmount;
        payable(msg.sender).transfer(etherAmount);

        emit Burn(msg.sender, etherAmount, _value);

        return true;
    }
```

Code Fragment 23: withdraw method of InsuranceLogic

```
function getQuote(uint256 T, string memory station, uint256 l) public view returns (
    uint256) {
        uint256 p = pricingLogic.getProbability(T, station);
        uint256 pp = (l * p) / factor;
        uint256 cp = (pp * (factor + eta)) / factor;
        return cp;
    }
```

Code Fragment 24: getQuote method of InsuranceLogic

```
function underwritePolicy(uint256 T, string memory station,  uint256 l) public payable {
        uint256 p = pricingLogic.getProbability(T, station);
        uint256 pp = (l * p) / factor;
        uint256 cp = (pp * (factor + eta)) / factor;
        require(msg.value == cp, "Incorrect premium amount sent");
        Nt++;
        (bool success, ) = payable(address(this)).call{value: msg.value}("");
        require(success, "Failed to transfer premium to reserve");
        modelPointsLogic.addInsuranceContract(station, T, p, l);
        mu += p * l * eta / factor / factor;
        l_tot += l;

        Nt_MP_active = modelPointsLogic.Nt_MP_active();
        if (Nt_MP_active < Nt_MP_active_min) {
            SCR = l_tot;
            MCR = l_tot;
        }
        else {
            uint256 sn2 = modelPointsLogic.sn2();
            sn = sqrt(sn2);
            uint256 gamma1_unormalized = modelPointsLogic.gamma1_unormalized();
            gamma1 = gamma1_unormalized / sn / sn / sn;

            SCR = sn * qAlphaSCR / factor + sn * gamma1 * qAlphaSCR * qAlphaSCR / factor
                / factor / 6 - sn * gamma1 / 6 - mu;
            MCR = sn * qAlphaMCR / factor + sn * gamma1 * qAlphaMCR * qAlphaMCR / factor
                / factor / 6 - sn * gamma1 / 6 - mu;
        }
        uint8 status = 0;
        InsuranceContract memory policy = InsuranceContract({
            customer: msg.sender,
            T: T,
            station: stringToBytes32(station), // Description of the insured event
            l: l,
            p:p ,
            eta: eta,
            refund:cp,
            status: status});
```


Code Fragment 25: Part 1 of underwritePolicy method of InsuranceLogic

```
    if (Xt <= SCR) {
        status = 3;
        policy.status = status;
        modelPointsLogic.removeInsuranceContract(Nt, "cancelled");
        sendPayout(payable(msg.sender), cp);
        mu -= p * l * eta / factor / factor;
        l_tot -= l;
        Nt_MP_active = modelPointsLogic.Nt_MP_active();
        if (Nt_MP_active < Nt_MP_active_min) {
            SCR = l_tot;
            MCR = l_tot;
        }
        else {
            uint256 sn2 = modelPointsLogic.sn2();
            sn = sqrt(sn2);
            uint256 gamma1_unormalized = modelPointsLogic.gamma1_unormalized();
            gamma1 = gamma1_unormalized / sn / sn / sn;
            SCR = sn * qAlphaSCR / factor + sn * gamma1 * qAlphaSCR * qAlphaSCR /
                factor / factor / 6 - sn * gamma1 / 6 - mu;
            MCR = sn * qAlphaMCR / factor + sn * gamma1 * qAlphaMCR * qAlphaMCR /
                factor / factor / 6 - sn * gamma1 / 6 - mu;
            }
    }
    insuranceContracts[Nt] = policy;
    emit InsuranceUnderwritten(Nt, msg.sender, T, stringToBytes32(station), l, cp,
        status, SCR, MCR);
}
```

Code Fragment 26: Part 2 of underwritePolicy method of InsuranceLogic

```
function settle(uint256 contractId, uint256 Q_observed) public {
        InsuranceContract storage policy = insuranceContracts[contractId];
        require(policy.customer != address(0), "Invalid contract ID");
        require(msg.sender == owner, "Not authorized");
        require(policy.status == 0, "Contract is not active");

        modelPointsLogic.removeInsuranceContract(contractId, "closed");
        mu -= policy.p * policy.l * policy.eta / factor / factor;
        l_tot -= policy.l;
        Nt_MP_active = modelPointsLogic.Nt_MP_active();
        if (Nt_MP_active < Nt_MP_active_min) {
            SCR = l_tot;
            MCR = l_tot;
        }
        else {
            uint256 sn2 = modelPointsLogic.sn2();
            sn = sqrt(sn2);
            uint256 gamma1_unormalized = modelPointsLogic.gamma1_unormalized();
            gamma1 = gamma1_unormalized / sn / sn / sn;

            SCR = sn * qAlphaSCR / factor + sn * gamma1 * qAlphaSCR * qAlphaSCR / factor
                / factor / 6 - sn * gamma1 / 6 - mu;
            MCR = sn * qAlphaMCR / factor + sn * gamma1 * qAlphaMCR * qAlphaMCR / factor
                / factor / 6 - sn * gamma1 / 6 - mu;
            }
        if (Q_observed >= 5) {
            sendPayout(payable(policy.customer), policy.l);
            policy.status = 1; // Settled with compensation
            updateReserveAfterSettlement(policy.l, policy.p, policy.eta, true);
            emit ClaimSettled(contractId, policy.customer, true, SCR, MCR);
        }
```

Code Fragment 27: Part 1 of `settle` method of InsuranceLogic

```
    else {
        updateReserveAfterSettlement(policy.l, policy.p, policy.eta, false);
        policy.status = 2; // Settled without compensation
        emit ClaimSettled(contractId, policy.customer, false, SCR, MCR);

    }
    if (Xt < MCR) {
        refundPremiumToPolicyholders();
        if (Xt > 0){
            redistributeToInvestors();
        }
        SCR=0;
        MCR=0;
        sn=0;
        mu=0;
        gamma1=0;
        rt=10000;
        Xt=0;
        Nt_MP_active=0;
        modelPointsLogic.cancelAllActiveContracts();
    }

}
```

Code Fragment 28: Part 2 of `settle` method of `InsuranceLogic`

```
function updateReserveAfterSettlement(uint256 l, uint256 p, uint256 eta_S, bool
    settlement) internal {
        uint256 pp = (l * p) / factor;
        uint256 cp = (pp * (factor + eta_S)) / factor;
        if (settlement) {
            Xt += cp;
            Xt -= l;

        } else {
            Xt += cp;
        }
        rt = totalSupply > 0 ? (Xt * (factor)) / totalSupply : 1;
    }
```

Code Fragment 29: updateReserveAfterSettlement method of InsuranceLogic

```
function refundPremiumToPolicyholders() internal {
        // require(Xt < MCR, "Reserve is above MCR");
        Xt = address(this).balance;
        require(Xt > 0, "Nothing to refund");
        uint256 TotalRefund = 0;
        uint256 ActivePolicyCount = 0;

        // Calculate total refund amount for active policies
        for (uint256 i = 1; i <= Nt; i++) {
            if (insuranceContracts[i].status == 0) { // Active contract
                TotalRefund += insuranceContracts[i].refund;
                ActivePolicyCount++;
            }
        }

        require(TotalRefund > 0, "No active policies to refund");

        // Check if the reserve is sufficient for full refunds
        if (Xt >= TotalRefund) {
            // Fully refund all policyholders
            for (uint256 i = 1; i <= Nt; i++) {
                if (insuranceContracts[i].status == 0) { // Active contract
                    uint256 refundAmount = insuranceContracts[i].refund;
                    insuranceContracts[i].status = 3; // Mark as refunded
                    modelPointsLogic.removeInsuranceContract(i, "cancelled");
                    sendPayout(payable(insuranceContracts[i].customer), refundAmount);
                    Xt -= refundAmount;
                }
            }
        } else {
            // Prorated refunds
            for (uint256 i = 1; i <= Nt; i++) {
                if (insuranceContracts[i].status == 0) { // Active contract
                    uint256 proratedRefund = (insuranceContracts[i].refund * Xt) /
                        TotalRefund;
                    insuranceContracts[i].status = 3; // Mark as refunded
                    modelPointsLogic.removeInsuranceContract(i, "cancelled");
                    sendPayout(payable(insuranceContracts[i].customer), proratedRefund);
                }
            }
            Xt = 0; // All reserve used
        }
```

```
function redistributeToInvestors() internal {
        Xt = address(this).balance;
        require(Xt > 0, "Nothing to distribute");
        for (uint256 i = 0; i < investorAddresses.length; i++) {
            address investor = investorAddresses[i];
            uint256 investorShare = (Yt[investor] * Xt) / totalSupply;
            Yt[investor] = 0;
            if (investorShare > 0) {
                sendPayout(payable(investor), investorShare);
            }
        }
        Xt = 0; // All reserve distributed
        totalSupply = 0;
        delete investorAddresses;
    }
```

Code Fragment 31: redistributeToInvestors method of InsuranceLogic

```
function sqrt(uint256 y) public pure returns (uint256 z) {
        if (y > 3) {
            z = y;
            uint256 x = y / 2 + 1;
            while (x < z) {
                z = x;
                x = (y / x + x) / 2;
            }
        } else if (y != 0) {
            z = 1;
        }
    }
```

Code Fragment 32: redistributeToInvestors method of InsuranceLogic

```
function sendPayout(address payable customer, uint256 l) internal {
        require(address(this).balance >= l, "Insufficient reserve for payout");
        (bool success, ) = customer.call{value: l}("");
        require(success, "Failed to send payout to policyholder");
    }
```

Code Fragment 33: sendPayout method of InsuranceLogic

```
function stringToBytes32(string memory source) public pure returns (bytes32 result) {
        bytes memory tempEmptyStringTest = bytes(source);
        if (tempEmptyStringTest.length == 0) {
            return 0x0;
        }

        assembly {
            result := mload(add(source, 32))
        }
    }
```

Code Fragment 34: stringToBytes32 method of InsuranceLogic